

Agentic AI Pipeline for Coverage-Directed Unit Test Generation

Sanyam Verma IMT2023040

Gautam Kappagal IMT2023082

github.com/sanyamvermaa/AI-Assisted-Unit-Testing-Pipeline

Testing requirement pursued: (1) user-specified coverage criterion – statement coverage and branch coverage, target 100%.

1 Pipeline, Dataset and Test-Generator Functionality

Dataset. MBPP ([google-research-datasets/mbpp](https://github.com/google-research-datasets/mbpp)), train split, five problems (task IDs 601, 602, 604, 605×2). Task ID 603 (“lucid number”) was skipped because all tested free models consistently returned syntactically invalid Python for that prompt; the pipeline’s `-skip-mbpp-ids` flag substituted the next available problem. MBPP 605 was run twice independently (different random seeds / model assignments), providing a comparison of two generated implementations of the same specification. Only the natural-language task is given to the pipeline; MBPP’s reference code and tests are stored alongside for traceability.

MBPP ID	Task
601	Write a function to find the longest chain which can be formed from the given set of pairs.
602	Write a python function to find the first repeated character in a given string.
604	Write a function to reverse words in a given string.
605 (trial A)	Write a function to check if the given integer is a prime number.
605 (trial B)	Write a function to check if the given integer is a prime number.

Table 1: MBPP problems used (five pipeline runs).

Agents. The pipeline is plain Python (one class per agent, a loop in `pipeline.py`, direct HTTPS calls to OpenRouter); no RAG, no agent framework and no chain-of-thought prompting is used.

1. **Code Generator Agent** (LLM): MBPP task → `solution.py`.
2. **Test Generator Agent** (LLM): task + generated code + coverage criterion → `test_solution.py` (Python `unittest`).
3. **Test Executor Agent** (deterministic): runs the suite under `coverage.py` in branch mode and returns the verdict (PASS/FAIL/TIMEOUT), statement %, branch % and the uncovered lines/branches. Execution and coverage are measured by tools rather than an LLM so the verdict cannot be hallucinated.

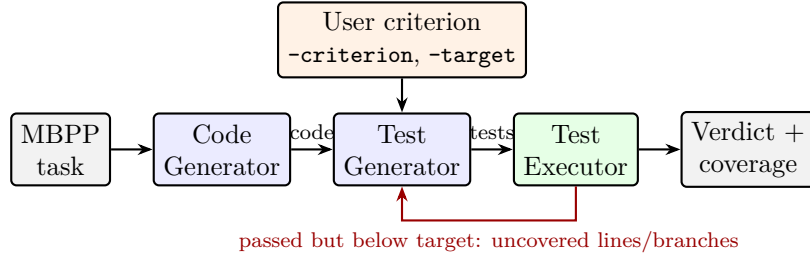


Figure 1: Pipeline. Blue: LLM agents; green: deterministic executor agent.

Test-generator functionality. The user selects the criterion (**statement+branch** (default), **statement** or **branch**) and a target. The prompt asks for representative, boundary and edge inputs, both outcomes of every conditional, and zero/one-or-more loop iterations. Coverage is computed as

$$\text{Stmt\%} = \frac{\text{statements} - \text{missing}}{\text{statements}} \times 100, \quad \text{Branch\%} = \frac{\text{covered branches}}{\text{total branches}} \times 100.$$

If a suite passes but misses the target, the executor’s list of uncovered statements and branch outcomes is sent back to the Test Generator to extend the suite (at most 2 rounds). Failing suites are not sent back, since a failure may reveal a real defect in the code. Generated output is rejected unless it parses (`ast.parse`), and test modules must use `unittest` and contain a `test_` method. On a validation failure the agent automatically retries with the next available free model, excluding the model that returned invalid output.

2 Prompts and Settings

Each agent fills a fixed template and sends it as a single **user** message (no system message, no history). The user input is the MBPP slice (`-n`) and the testing requirement (`-criterion, -target`).

```

You are the Code Generator Agent in an automated software-testing experiment.

Programming problem:
{task}

Generate a correct Python implementation for the problem.

Strict output rules:
- Return ONLY executable Python source code.
- Do not use Markdown code fences.
- Do not include explanations.
- Do not include JSON.
- Do not include safety classifications.
- Do not include analysis.
- Define the function(s) required by the problem.
- Keep the implementation concise and self-contained.
  
```

Listing 1: Code Generator prompt.

```

You are the Test Generator Agent in an AI-assisted unit-testing pipeline.

Programming problem:
{task}

Generated Python implementation:
{implementation}

Create a complete Python unittest module named test_solution.py.

Test-design requirements:
  
```

1. Use Python’s built-in unittest framework.
2. Import the function(s) from solution.py.
3. Include normal/representative cases.
4. Include boundary cases where meaningful.
5. Include edge cases such as empty, singleton, negative, duplicate, or unusual inputs when applicable.
6. For each important conditional, exercise both outcomes.
7. For loops, exercise zero iterations and one-or-more iterations when meaningful.
8. Use strong assertions with expected outputs.
9. Do not test implementation details that are irrelevant to the problem specification.
10. {criterion_text}

Strict output rules:

- Return ONLY executable Python source code.
- Do not use Markdown code fences.
- Do not include explanations.
- Do not include JSON.
- Do not include safety classifications.

Listing 2: Test Generator prompt. Item 10 depends on the chosen criterion; for `statement+branch` it is “Aim for high statement and branch coverage.”

The coverage-feedback prompt repeats the task, the code with line numbers and the current tests, adds “Statements never executed: ...” and “Branch outcomes never taken: ...” from the executor, and asks for the complete updated module.

Parameter	Value
LLM provider	OpenRouter free models (auto-discovered; excluded per-call on validation failure)
Models used	nvidia/nemotron-3.5-lightning:free, qwen/qwen3.8-27b:free, nvidia/nemotron-3-ultra-550b-a55b:free, nvidia/nemotron-3-nano-omni-30b-a3b-reasoning:free
Temperature	0.2 (all LLM calls)
max_tokens	1500 (code), 2500 (tests)
Other sampling parameters	provider defaults
HTTP timeout	10s connect, 60s read
Test framework / coverage	unittest; coverage run -branch -source=solution
Executor timeout	180s per suite
Criterion / target	statement + branch, 100%

Table 2: Pipeline settings.

3 Format of Code, Tests and Verdict

Code: generated/problem_N/solution.py, a plain Python module with the required function(s).
Tests: generated/problem_N/test_solution.py, a `unittest.TestCase` class with one `test_*` method per scenario using `assertEqual/assertTrue/...` with explicit expected values. **Verdict:** a JSON record saved in `experiment.json`:

```
{ "verdict": "PASS", "passed": true, "tests_run": 16,
  "statement_coverage": 100.0, "branch_coverage": 100.0,
  "missing_lines": [], "missing_branches": [],
  "execution_time_seconds": 0.1, "stderr": "Ran 16 tests in 0.001s OK" }
```

Generated code and test cases

```
def find_longest_chain(pairs):
    pairs.sort(key=lambda x: x[1])
    count = 0
    last_end = float('-inf')
    for a, b in pairs:
        if a > last_end:
            count += 1
            last_end = b
    return count
```

Listing 3: MBPP 601 – generated code

```
import unittest
from solution import find_longest_chain

class TestFindLongestChain(unittest.TestCase):

    def test_empty_list(self):
        self.assertEqual(find_longest_chain([]), 0)

    def test_singleton(self):
        self.assertEqual(find_longest_chain([[1, 2]]), 1)

    def test_basic_chain(self):
        pairs = [[1, 2], [3, 4], [5, 6]]
        self.assertEqual(find_longest_chain(pairs), 3)

    def test_no_chain_possible(self):
        pairs = [[1, 2], [2, 3]]
        self.assertEqual(find_longest_chain(pairs), 1)

    def test_negative_numbers(self):
        pairs = [[-3, -1], [-2, 0], [1, 2]]
        self.assertEqual(find_longest_chain(pairs), 2)

    def test_duplicate_pairs(self):
        pairs = [[1, 2], [1, 2], [1, 2]]
        self.assertEqual(find_longest_chain(pairs), 1)

    def test_unordered_input(self):
        pairs = [[3, 4], [1, 2]]
```

Listing 4: MBPP 601 – generated tests

```
def first_repeated_char(s):
    seen = set()
    for ch in s:
        if ch in seen:
            return ch
        seen.add(ch)
    return None
```

Listing 5: MBPP 602 – generated code

```
import unittest

from solution import first_repeated_char

class TestFirstRepeatedChar(unittest.TestCase):
    def test_empty_string_returns_none(self):
        self.assertIsNone(first_repeated_char(""))
```

```

def test_single_character_returns_none(self):
    self.assertIsNone(first_repeated_char("a"))

def test_no_repeated_characters_returns_none(self):
    self.assertIsNone(first_repeated_char("abc"))

def test_immediate_repeat_returns_character(self):
    self.assertEqual(first_repeated_char("aa"), "a")

def test_first_repeated_is_earliest_second_occurrence(self):
    self.assertEqual(first_repeated_char("abcabc"), "a")

def test_repeat_of_later_character_before_earlier_character(self):
    self.assertEqual(first_repeated_char("abcba"), "b")

def test_multiple_repeats_returns_first_repeated(self):
    self.assertEqual(first_repeated_char("aabbcc"), "a")

def test_all_same_returns_first_char(self):
    self.assertEqual(first_repeated_char("aaaa"), "a")

def test_repeat_of_space(self):
    self.assertEqual(first_repeated_char("a b a"), " ")

def test_repeat_of_digit(self):
    self.assertEqual(first_repeated_char("121"), "1")

def test_repeat_of_unicode(self):
    self.assertEqual(first_repeated_char("éé"), "é")

def test_mixed_case_distinct_returns_none(self):
    self.assertIsNone(first_repeated_char("AbC"))

def test_mixed_case_repeat(self):
    self.assertEqual(first_repeated_char("aAa"), "a")

def test_two_chars_one_repeat(self):
    self.assertEqual(first_repeated_char("bb"), "b")

def test_long_no_repeat(self):
    self.assertIsNone(first_repeated_char("abcdefg"))

def test_repeat_at_end(self):
    self.assertEqual(first_repeated_char("abcdefa"), "a")

if __name__ == "__main__":
    unittest.main()

```

Listing 6: MBPP 602 – generated tests

```

def reverse_words(s: str) -> str:
    return " ".join(s.split()[::-1])

```

Listing 7: MBPP 604 – generated code

```

import unittest

from solution import reverse_words

class TestReverseWords(unittest.TestCase):
    def test_empty_string(self):
        self.assertEqual(reverse_words(""), "")

```

```

def test_single_word(self):
    self.assertEqual(reverse_words("hello"), "hello")

def test_two_words(self):
    self.assertEqual(reverse_words("hello world"), "world hello")

def test_multiple_words(self):
    self.assertEqual(
        reverse_words("the quick brown fox"),
        "fox brown quick the"
    )

def test_leading_and_trailing_spaces(self):
    self.assertEqual(reverse_words(" hello world "), "world hello")

def test_multiple_spaces_between_words(self):
    self.assertEqual(reverse_words("hello  world"), "world hello")

def test_whitespace_only_string(self):
    self.assertEqual(reverse_words(" \t\n "), "")

def test_mixed_whitespace_separators(self):
    self.assertEqual(reverse_words("a\tb\nc"), "c b a")

def test_words_with_punctuation(self):
    self.assertEqual(reverse_words("hello, world!"), "world! hello,")

def test_unicode_words(self):
    self.assertEqual(reverse_words("héllo wörld"), "wörld héllo")

def test_numeric_words(self):
    self.assertEqual(reverse_words("1 2 3"), "3 2 1")

def test_single_character_words(self):
    self.assertEqual(reverse_words("a b c"), "c b a")

def test_duplicate_words(self):
    self.assertEqual(reverse_words("same same same"), "same same same")

def test_returns_string(self):
    self.assertIsInstance(reverse_words("hello world"), str)

if __name__ == "__main__":
    unittest.main()

```

Listing 8: MBPP 604 – generated tests

```

def is_prime(n):
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

```

Listing 9: MBPP 605 trial A – generated code

```
import unittest
```

```

from solution import is_prime

class TestIsPrime(unittest.TestCase):
    def test_non_positive(self):
        self.assertFalse(is_prime(0))
        self.assertFalse(is_prime(1))
        self.assertFalse(is_prime(-1))
        self.assertFalse(is_prime(-10))

    def test_small_numbers(self):
        self.assertTrue(is_prime(2))
        self.assertTrue(is_prime(3))
        self.assertFalse(is_prime(4))
        self.assertFalse(is_prime(6))
        self.assertFalse(is_prime(8))
        self.assertFalse(is_prime(9))
        self.assertFalse(is_prime(15))

    def test_prime_numbers(self):
        primes = [5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,83,89,97]
        for p in primes:
            with self.subTest(p=p):
                self.assertTrue(is_prime(p))

    def test_composite_numbers(self):
        composites =
[4,6,8,9,10,12,14,15,16,18,20,21,22,24,25,26,27,28,30,32,33,34,35,36,38,39,40,42,44,45,46,48,49,50,51,52,54,5
        for n in composites:
            with self.subTest(n=n):
                self.assertFalse(is_prime(n))

    def test_boundary_values(self):
        self.assertFalse(is_prime(0))
        self.assertFalse(is_prime(1))
        self.assertTrue(is_prime(2))
        self.assertTrue(is_prime(3))
        self.assertFalse(is_prime(4))

    def test_negative_numbers(self):
        for n in [-1, -2, -3, -10, -100]:
            self.assertFalse(is_prime(n))

    def test_large_numbers(self):
        primes =
[101,103,107,109,113,127,131,137,139,149,151,157,163,167,173,179,181,191,193,197,199]
        for p in primes:
            with self.subTest(p=p):
                self.assertTrue(is_prime(p))

        self.assertFalse(is_prime(100))
        self.assertFalse(is_prime(1024))
        self.assertFalse(is_prime(1048576))
        self.assertFalse(is_prime(999984))

    def test_loop_iteration_cases(self):
        # zero iterations
        self.assertTrue(is_prime(2))
        self.assertTrue(is_prime(3))
        self.assertTrue(is_prime(5))

        # one or more iterations
        self.assertFalse(is_prime(25)) # 5*5, loop runs once
        self.assertFalse(is_prime(121)) # 11*11, loop runs twice
        self.assertFalse(is_prime(221)) # 13*17, loop runs multiple times

```

```
if __name__ == '__main__':
    unittest.main()
```

Listing 10: MBPP 605 trial A – generated tests

```
def is_prime(n):
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    i = 3
    while i * i <= n:
        if n % i == 0:
            return False
        i += 2

    return True
```

Listing 11: MBPP 605 trial B – generated code

```
import unittest
from solution import is_prime

class TestIsPrime(unittest.TestCase):
    def test_negative_numbers(self):
        self.assertFalse(is_prime(-1))
        self.assertFalse(is_prime(-2))
        self.assertFalse(is_prime(-10))
        self.assertFalse(is_prime(-100))

    def test_zero_and_one(self):
        self.assertFalse(is_prime(0))
        self.assertFalse(is_prime(1))

    def test_two(self):
        self.assertTrue(is_prime(2))

    def test_even_numbers_greater_than_two(self):
        for n in [4, 6, 8, 10, 12, 14, 16, 18, 20, 100]:
            self.assertFalse(is_prime(n), f"{n} should not be prime")

    def test_small_primes(self):
        primes = [3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79,
83, 89, 97]
        for p in primes:
            self.assertTrue(is_prime(p), f"{p} should be prime")

    def test_small_non_primes_odd(self):
        non_primes = [9, 15, 21, 25, 27, 33, 35, 39, 45, 49, 51, 55, 57, 63, 65, 69, 75, 77, 81,
85, 87, 91, 93, 95, 99]
        for n in non_primes:
            self.assertFalse(is_prime(n), f"{n} should not be prime")

    def test_large_prime(self):
        self.assertTrue(is_prime(9973))

    def test_large_non_prime(self):
        self.assertFalse(is_prime(9999))
        self.assertFalse(is_prime(10000))
        self.assertFalse(is_prime(10001))
```

```

def test_loop_zero_iterations(self):
    # n=3 -> i=3, 3*3=9 > 3, loop body never executes
    self.assertTrue(is_prime(3))

def test_loop_one_iteration(self):
    # n=9 -> i=3, 3*3=9 <=9, one iteration, returns False
    self.assertFalse(is_prime(9))

def test_loop_multiple_iterations(self):
    # n=25 -> i=3 (3*3=9<=25), i=5 (5*5=25<=25) -> two iterations
    self.assertFalse(is_prime(25))
    # n=49 -> i=3,5,7 -> three iterations
    self.assertFalse(is_prime(49))

if __name__ == "__main__":
    unittest.main()

```

Listing 12: MBPP 605 trial B – generated tests

4 Execution Results

MBPP ID	Tests generated	Verdict	Statement cov.	Branch cov.	Runtime (s)
601	7	PASS	100%	100%	281.0
602	16	PASS	100%	100%	69.1
604	14	PASS	100%	100%	71.9
605 (A)	8	PASS	100%	100%	262.7
605 (B)	11	PASS	100%	100%	242.1
Total	56	5/5	100%	100%	926.8

Table 3: Results of the Test Generator and Test Executor agents. Runtime includes LLM latency.

All five suites met the criterion on the first attempt, so no coverage-feedback round was needed. Test execution itself took under a second per problem; nearly all runtime was LLM latency on free-tier models (e.g. MBPP 601: 56.5s code generation, 224.0s test generation, <0.1s execution).

The two independent runs of MBPP 605 generated structurally different implementations (trial A uses the $6k \pm 1$ divisibility optimisation; trial B uses a simple odd-step loop) and different test suites (8 vs. 11 methods), both achieving 100% statement and branch coverage.

Limitations. 100% statement and branch coverage shows every statement and decision outcome was executed, not that the code is correct: the tests are written from the generated code, so a misunderstanding shared by both would still PASS. Task ID 603 (“lucid number”) was skipped because free models consistently returned syntactically invalid Python for it, suggesting the task description is ambiguous or too obscure for small free-tier models. Running MBPP’s reference tests or mutation testing would give an independent correctness signal.

5 Individual Contributions

Member	Contribution
Sanyam Verma (IMT2023040)	Selected the testing goal (coverage criterion) and dataset; built the Code Generator and Test Generator agents and the OpenRouter client; designed the test-generation prompt (boundary/edge cases, both outcomes of each decision, loop iterations) and the coverage-feedback loop; output validation of generated code and tests; per-call model exclusion on validation failure.
Gautam Kappagal (IMT2023082)	Built the Test Executor agent: running suites with <code>unittest</code> and <code>coverage.py</code> , statement/branch coverage computation, PASS/-FAIL/TIMEOUT verdicts; pipeline orchestration and result logging; ran the experiment, analysed the coverage results and limitations, and wrote the report.